

Data Structures Midterm Examination

Detailed Solutions Manual

Shahid Madani University of Azerbaijan

Question 1

Calculate the time complexity of the following code snippet:

```
for (i = 1, j = n; i <= j; i++, j--)  
    cout << i + j;
```

Solution:

The `for` loop initializes two pointers: i starting at 1 and j starting at n . In each iteration, i increments by 1 while j decrements by 1, decreasing the gap between them by 2 units per step. The loop continues as long as $i \leq j$, executing approximately $\lfloor n/2 \rfloor$ times.

Since the statement inside the loop performs a constant number of operations $\mathcal{O}(1)$, the total time complexity is:

$$\Theta(n)$$

Question 2

Calculate the complexity $T(n)$ of the iterative Insertion Sort algorithm. Specify $T(n) \in \mathcal{O}(?)$, $T(n) \in \Omega(?)$, and $T(n) \in \Theta(?)$. Furthermore, sort the elements $[65, 3, 1, 2]$ using this algorithm step-by-step.

Solution:

A) Complexity Analysis of Insertion Sort:

- **Best-case complexity:** Occurs when the input array is already sorted. The inner `while` loop condition fails immediately on the first evaluation, executing only the outer loop $n - 1$ times. Thus, $T(n) \in \Omega(n)$.
- **Worst-case complexity:** Occurs when the input array is sorted in reverse order. The inner loop executes k times for each step k from 1 to $n - 1$. The total number of inner loop executions is $\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \mathcal{O}(n^2)$. Thus, $T(n) \in \mathcal{O}(n^2)$.
- **Theta Bound:** A single tight Θ bound does not exist for all cases simultaneously. However, for the worst-case scenario it is $\Theta(n^2)$, and for the best-case scenario it is $\Theta(n)$.

B) Trace for Array $A = [65, 3, 1, 2]$:

- **Initial Array:** [65, 3, 1, 2]
- **Iteration $k = 1$ ($key = 3$):** $65 > 3$, shift 65 to the right $\rightarrow [3, 65, 1, 2]$
- **Iteration $k = 2$ ($key = 1$):** $65 > 1$ and $3 > 1$, shift both to the right $\rightarrow [1, 3, 65, 2]$
- **Iteration $k = 3$ ($key = 2$):** $65 > 2$ and $3 > 2$, shift both; $1 < 2$, insert 2 $\rightarrow [1, 2, 3, 65]$

Question 3

Find the complexity $T(n)$ of the recursive Tower of Hanoi algorithm, and list the step-by-step output trace for $n = 4$.

Solution:

A) Time Complexity Analysis: The recurrence relation for the Tower of Hanoi algorithm is given by:

$$T(n) = 2T(n-1) + 1, \quad T(1) = 1$$

Solving this recurrence by repeated substitution:

$$T(n) = 2^n - 1 \implies \Theta(2^n)$$

B) Output Trace for $n = 4$ (Hanoi(A, B, C, 4)): For $n = 4$, the algorithm requires $2^4 - 1 = 15$ moves:

- | | | |
|----------------------|-----------------------|-----------------------|
| 1. A $\rightarrow$ B | 6. C $\rightarrow$ B | 11. C $\rightarrow$ A |
| 2. A $\rightarrow$ C | 7. A $\rightarrow$ B | 12. B $\rightarrow$ C |
| 3. B $\rightarrow$ C | 8. A $\rightarrow$ C | 13. A $\rightarrow$ B |
| 4. A $\rightarrow$ B | 9. B $\rightarrow$ C | 14. A $\rightarrow$ C |
| 5. C $\rightarrow$ A | 10. B $\rightarrow$ A | 15. B $\rightarrow$ C |

Question 4

Provide a recursive algorithm that finds both the minimum and maximum elements among n numbers using at most $\frac{3n}{2} - 2$ comparisons.

Solution:

To achieve exactly $\frac{3n}{2} - 2$ comparisons, we employ a ****Divide and Conquer**** strategy:

```
void findMinMax(int A[], int low, int high, int &min, int &max) {
    if (low == high) {
        min = max = A[low];
    }
```

```

        return;
    }
    if (high == low + 1) {
        if (A[low] < A[high]) {
            min = A[low]; max = A[high];
        } else {
            min = A[high]; max = A[low];
        }
        return;
    }
    int mid = (low + high) / 2;
    int min1, max1, min2, max2;
    findMinMax(A, low, mid, min1, max1);
    findMinMax(A, mid + 1, high, min2, max2);
    min = (min1 < min2) ? min1 : min2;
    max = (max1 > max2) ? max1 : max2;
}

```

Question 5

Solve the following recurrence relation using the **Recursion Tree** method:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(\frac{n}{2}) + \log n & n \geq 2 \end{cases}$$

Solution:

Analyzing node costs level-by-level using a recursion tree:

- **Level 0 (Root):** Cost = $\log n$
- **Level 1:** 2 subproblems of size $n/2$. Total cost = $2 \log(n/2) = 2(\log n - 1)$
- **Level i :** 2^i subproblems of size $n/2^i$. Total cost = $2^i \log(n/2^i) = 2^i(\log n - i)$
- **Tree Height:** $k = \log_2 n$. At the leaf level ($i = k$), there are $2^k = n$ leaves each taking $\Theta(1)$ cost, yielding a leaf cost of $\Theta(n)$.

Summing costs across all levels:

$$T(n) = \sum_{i=0}^{k-1} 2^i(\log n - i) + \Theta(n)$$

$$T(n) = \log n \left(\sum_{i=0}^{k-1} 2^i \right) - \left(\sum_{i=0}^{k-1} i 2^i \right) + \Theta(n)$$

Using geometric and arithmetico-geometric series formulas:

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1 = n - 1$$

$$\sum_{i=0}^{k-1} i 2^i = (k - 2) 2^k + 2 = (\log n - 2)n + 2$$

Substituting back into the total sum:

$$T(n) = \log n(n - 1) - (n \log n - 2n + 2) + n = 3n - \log n - 2$$

The dominant term is $3n$, so the time complexity is:

$$\Theta(n)$$

Question 6

Solve the recurrence relation $T(n) = 2T(\frac{n}{2}) + n^2$ using the **Master Theorem**.

Solution:

Matching $T(n) = aT(\frac{n}{b}) + f(n)$ gives:

$$a = 2, \quad b = 2, \quad f(n) = n^2$$

We calculate $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.

Comparing $f(n) = n^2$ with n : Since $f(n) = \Omega(n^{1+\epsilon})$ for $\epsilon = 1 > 0$, **Case 3** of the Master Theorem applies.

Now we check the **Regularity Condition**: $af(\frac{n}{b}) \leq cf(n)$ for some constant $c < 1$:

$$2 \left(\frac{n}{2} \right)^2 = \frac{n^2}{2} \leq c \cdot n^2$$

This inequality holds for $c = \frac{1}{2} < 1$. Thus, the regularity condition is satisfied.

By Case 3 of the Master Theorem:

$$T(n) = \Theta(f(n)) = \Theta(n^2)$$

Question 7

Using the formal definition of Big-O, determine whether $2^{n+1} \in \mathcal{O}(2^n)$ is **True** or **False**.

Solution:

By formal definition, $f(n) \in \mathcal{O}(g(n))$ if there exist positive constants $c > 0$ and $n_0 > 0$ such that:

$$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

Here, $f(n) = 2^{n+1}$ and $g(n) = 2^n$. We rewrite $f(n)$:

$$2^{n+1} = 2 \cdot 2^n$$

By choosing $c = 2$ and $n_0 = 1$:

$$2 \cdot 2^n \leq 2 \cdot 2^n \quad \forall n \geq 1$$

This inequality holds for all $n \geq 1$. Therefore, the statement is **True**.